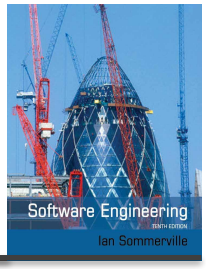


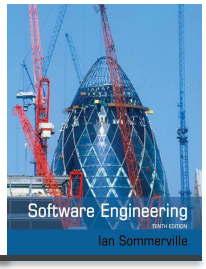
– Umbrella Activities & Software Processes Models

Umbrella Activity



- Any standard software process model would primarily consist of two types of activities:
- A set of framework activities, which are always applicable, regardless of the project type, and a set of umbrella activities, which are the non SDLC activities that span across the entire software development life cycle.

Typical activities in this category include:



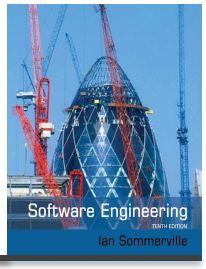
01. Software project tracking and control :

- Tracking and Control is the dual process of detecting when a project is drifting off-plan, and taking corrective action to bring the project back on track. But a successful project manager will also be able to tell when the plan itself is faulty, and even re-plan the project and its goals if necessary.

02. Formal technical reviews :

- This includes reviewing the techniques that has been used in the project.

Typical activities in this category include:



03. <u>Software quality assurance :</u>

- This is very important to ensure the quality measurement of each part to ensure them.

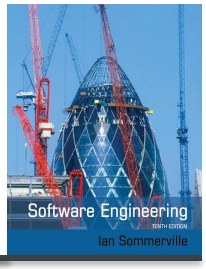
04. <u>Software configuration management :</u>

- In software engineering, software configuration management (SCM or S/W CM) is the task of tracking and controlling changes in thesoftware, part of the larger cross-disciplinary field of configuration management. SCM practices include revision control and the establishment of baselines.

05. <u>Document preparation and production :</u>

- All the project planning and other activities should be hardly copied and the production get started here.

Typical activities in this category include:



06. Reusability management :

- This includes the backing up of each part of the software project they can be corrected or any kind of support can be given to them later to update or upgrade the software at user/time demand.

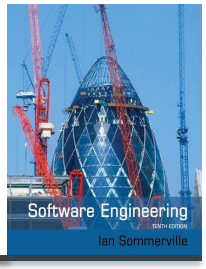
07. Measurement :

- This will include all the measurement of every aspects of the software project.

08. Risk management :

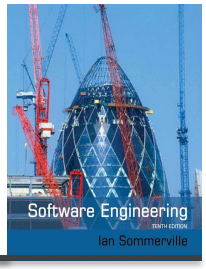
Risk management is a series of steps that help a software team to understand and manage uncertainty. It's a really good idea to identify it, assess its probability of occurrence, estimate its impact, and establish a contingency plan

The software process



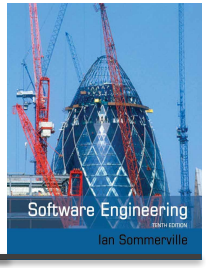
- A structured set of activities required to develop a software system.
- Many different software processes but all involve:
  - Specification – defining what the system should do;
  - Design and implementation – defining the organization of the system and implementing the system;
  - Validation – checking that it does what the customer wants;
  - Evolution – changing the system in response to changing customer needs.
- A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective.

Software process descriptions



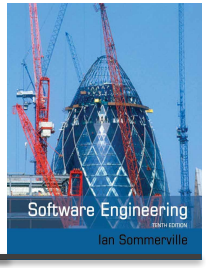
- When we describe and discuss processes, we usually talk about the activities in these processes such as specifying a data model, designing a user interface, etc. and the ordering of these activities.
- Process descriptions may also include:
  - Products, which are the outcomes of a process activity;
  - Roles, which reflect the responsibilities of the people involved in the process;
  - Pre- and post-conditions, which are statements that are true before and after a process activity has been enacted or a product produced.

Plan-driven and agile processes



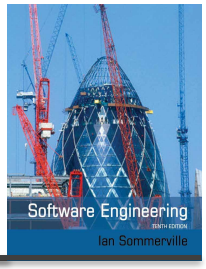
- Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan.
- In agile processes, planning is incremental and it is easier to change the process to reflect changing customer requirements.
- In practice, most practical processes include elements of both plan-driven and agile approaches.
- There are no right or wrong software processes.

Development Approaches

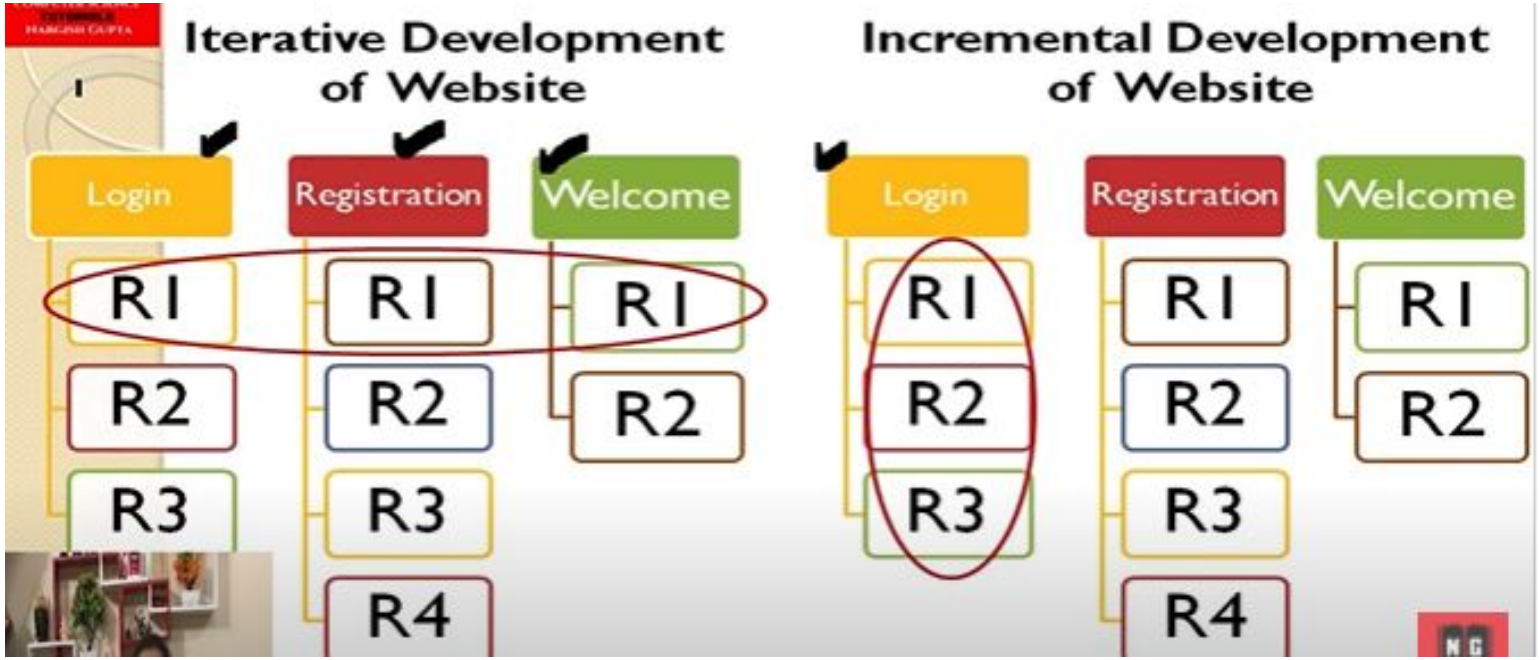


- Plan driven
- Incremental
- Iterative
- Evolutionary
- Component based

Incremental VS iterative



- Deployment after each increment but not after each iteration
- Deliver some requirements from each module in an iteration.
- We select one module and select all requirements of one module and deployed to cutomer



Incremental and Iterative Delivery

Iterative

- Develop through repeated cycles
- Start simple, expecting to change
- Used to find the right solution (fail early)
- Used to improve the candidate solution

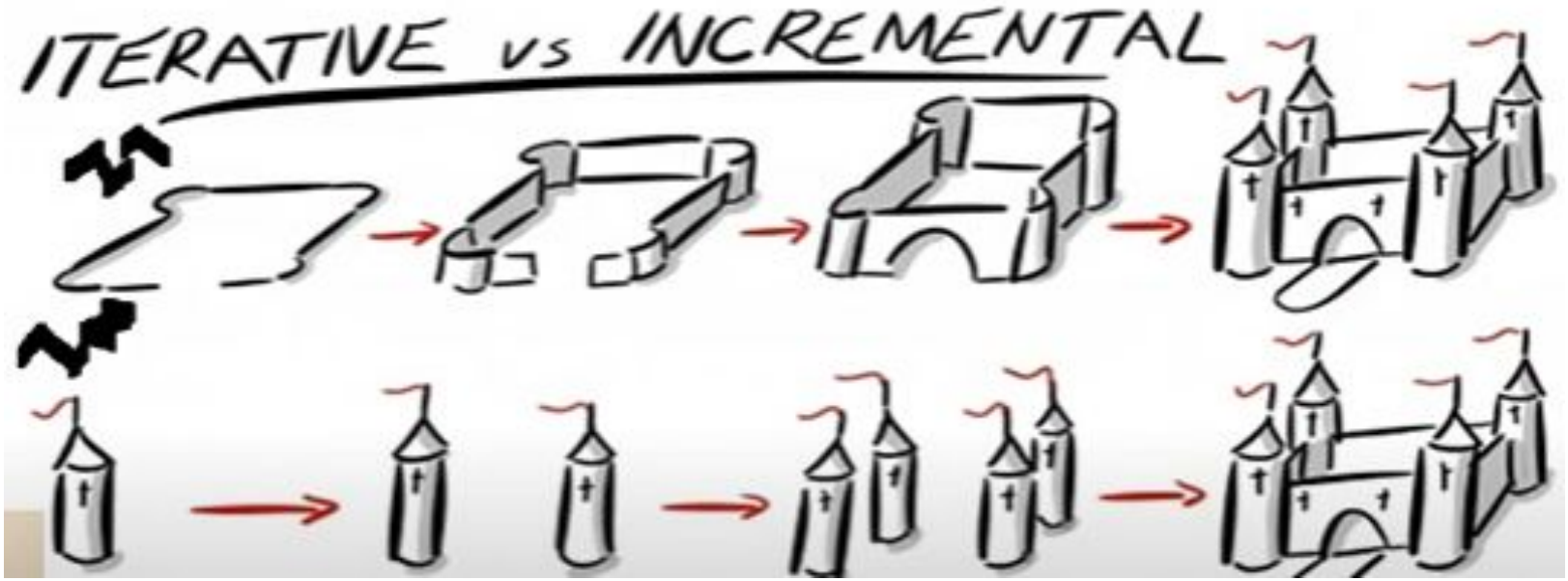


Incremental

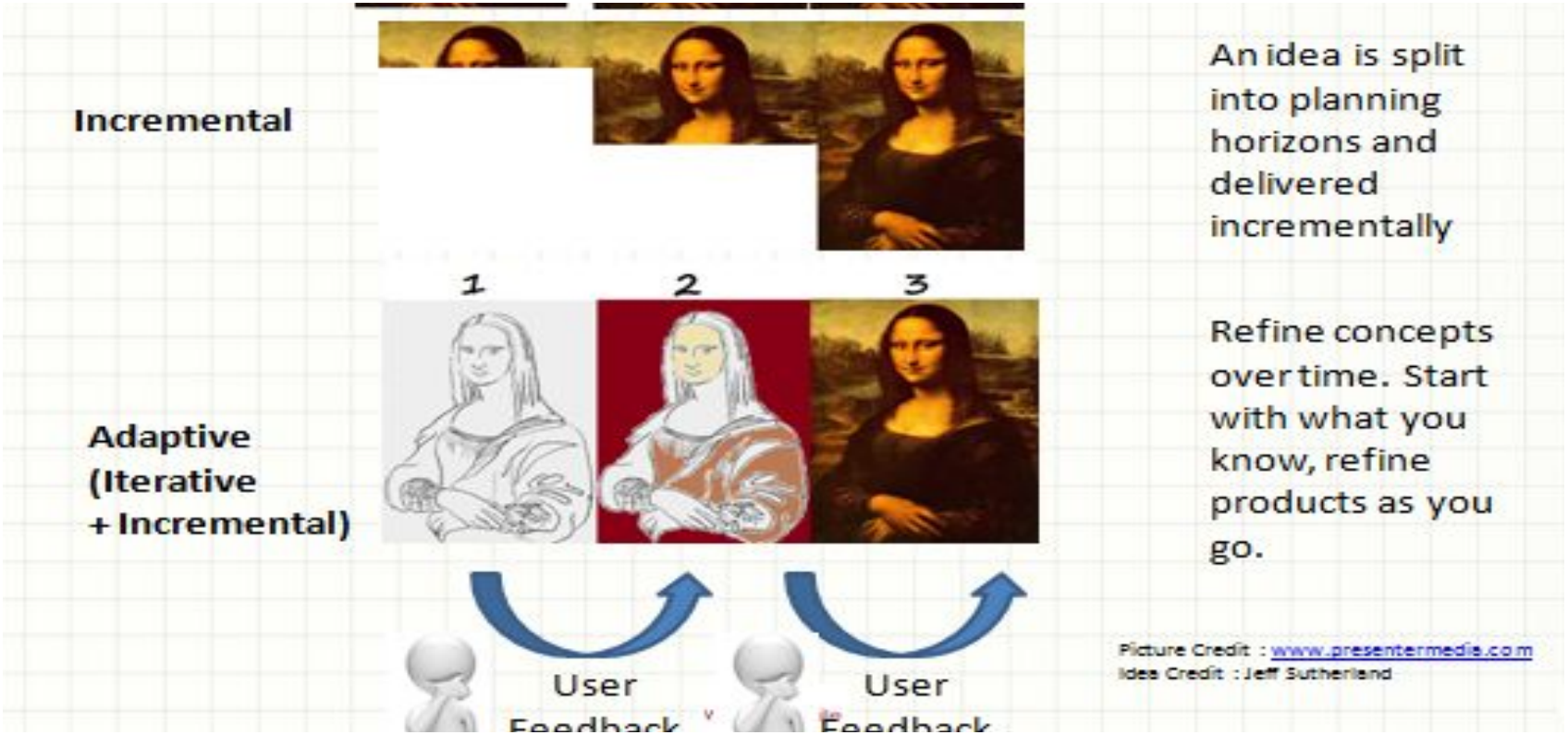
- Develop smaller portions at a time
- Gradually build up functionality
- Allows value to be delivered early

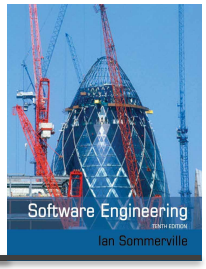


Incremental VS iterative –Fort example



Evolutionary





Software process models

Choosing a Software Process Model



Requirements Understanding



Expected lifeTime



Risk

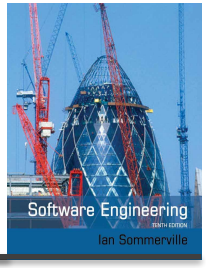


Schedule Constraints



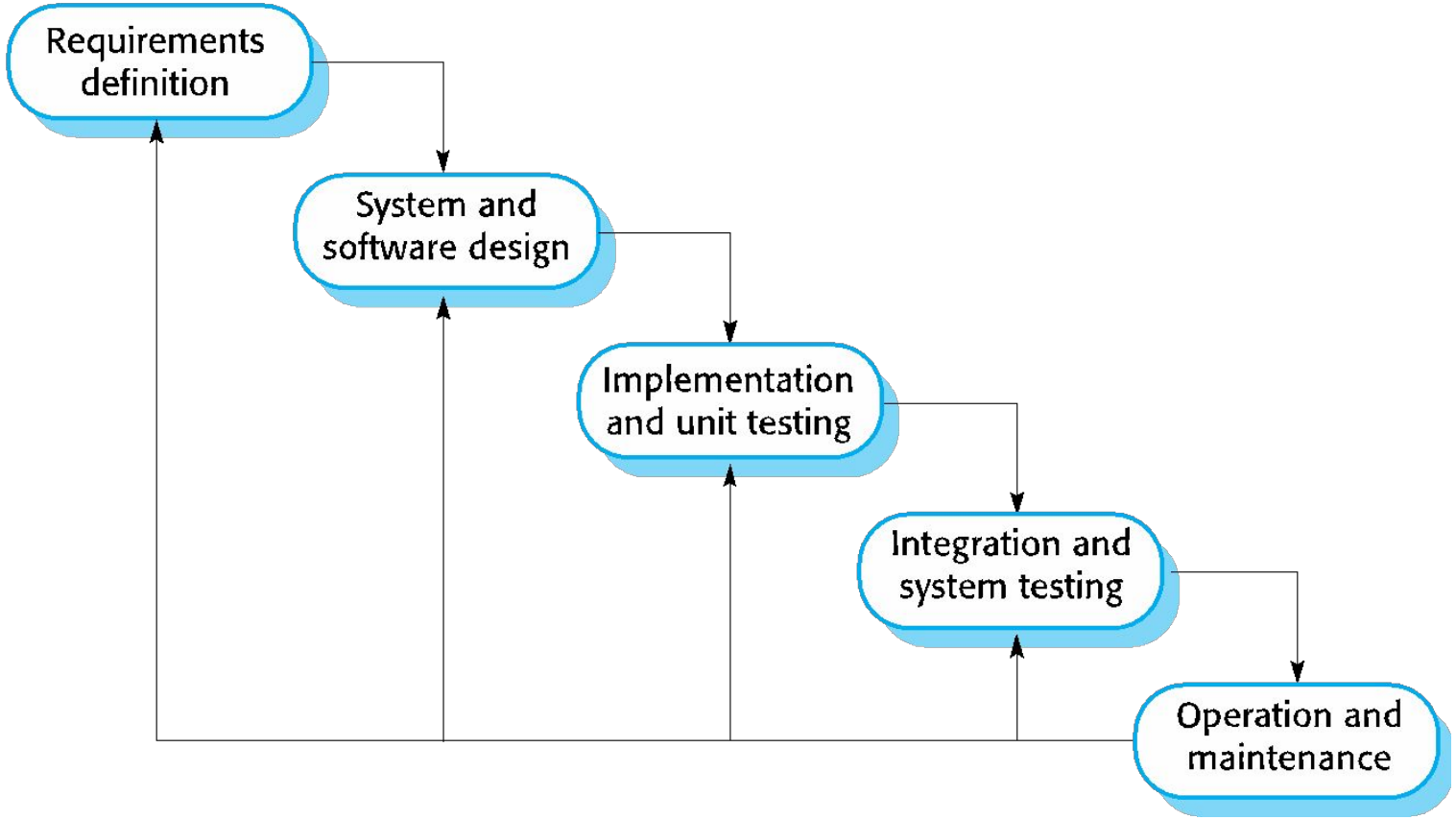
Interaction with management/customers

Software process models

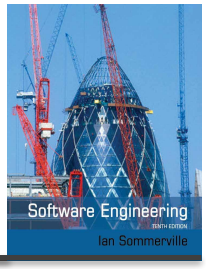


- The waterfall model
- Incremental development
- Spiral Model
- RAD
- Agile
- Scrum
- In practice, most large systems are developed using a process that incorporates elements from all of these models.

The waterfall model

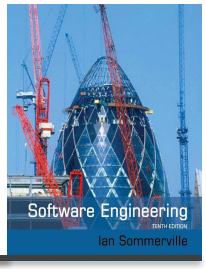


Waterfall model phases



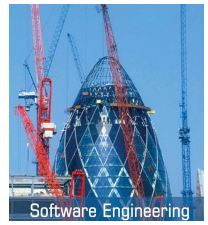
- There are separate identified phases in the waterfall model:
  - Requirements analysis and definition
  - System and software design
  - Implementation and unit testing
  - Integration and system testing
  - Operation and maintenance
- The main drawback of the **waterfall model** is the difficulty of accommodating change after the process is underway.
- In principle, a phase has to be complete before moving onto the next phase.

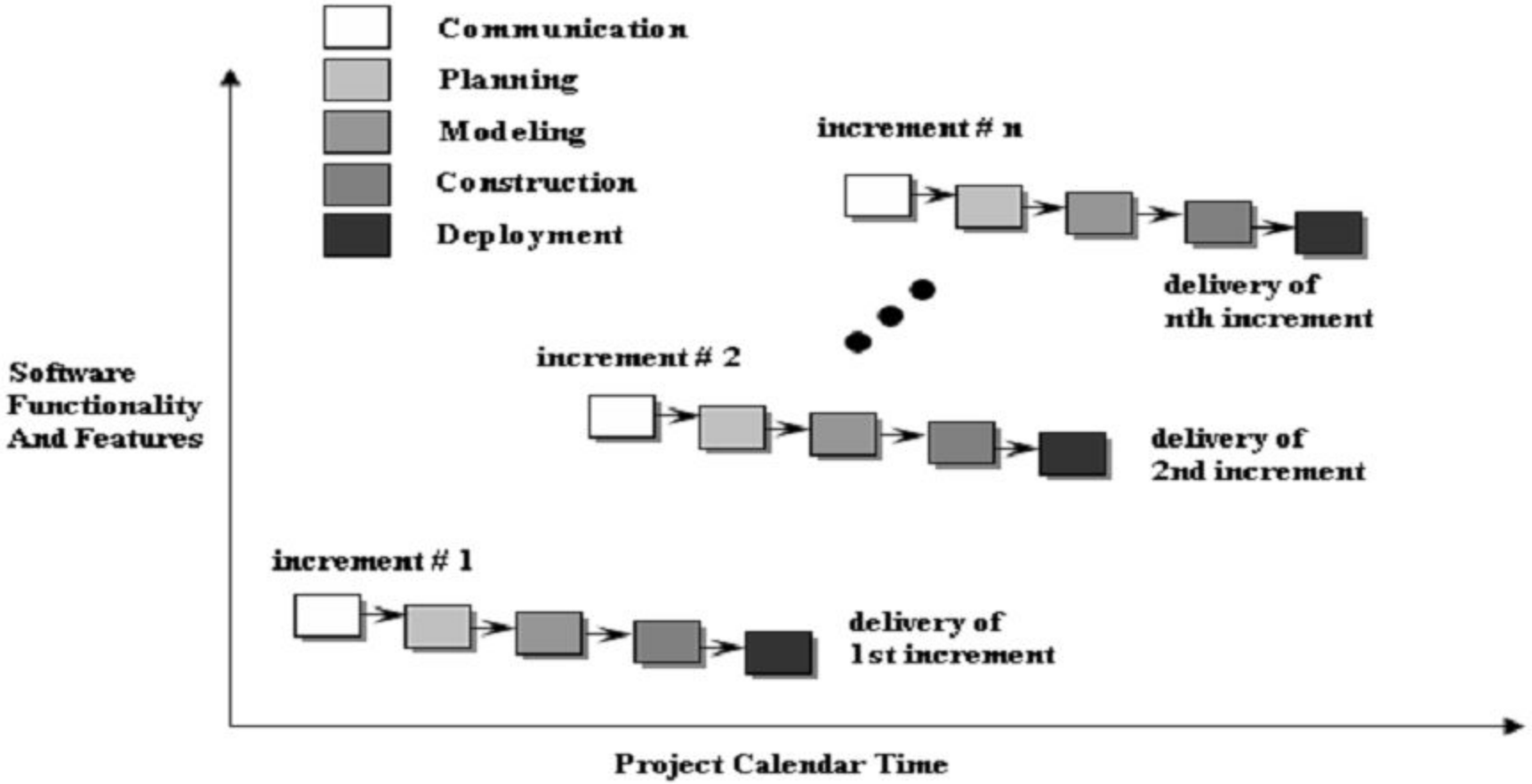
Waterfall model problems



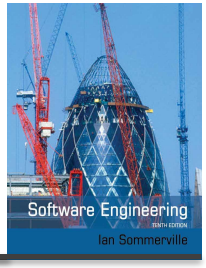
- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.
  - Therefore**, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.**
  - Few business systems have stable requirements.
- The waterfall model is mostly used for large systems engineering projects where a system is developed at several sites.
  - In those circumstances, the plan-driven nature of the waterfall model helps coordinate the work.

Incremental Model



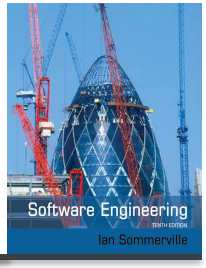


Incremental model



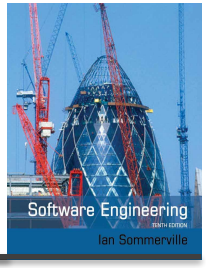
- There are many situations in which initial software requirements are reasonably well defined but the overall scope of the development effort prevents a purely linear process.
- The **incremental model** combines elements of the waterfall model applied in an iterative fashion.
- The incremental model applies linear sequences in a staggered fashion as calendar time progresses.
- Each linear sequence produces deliverable —incrementsǁ of the software.
- When an incremental model is used, the first increment is often a **core product**.
- The incremental process model, like prototyping and other evolutionary approaches, is iterative in nature.
- Unlike prototyping, the incremental model focuses on the delivery of an operational product with each increment.

Advantages and Disadvantages



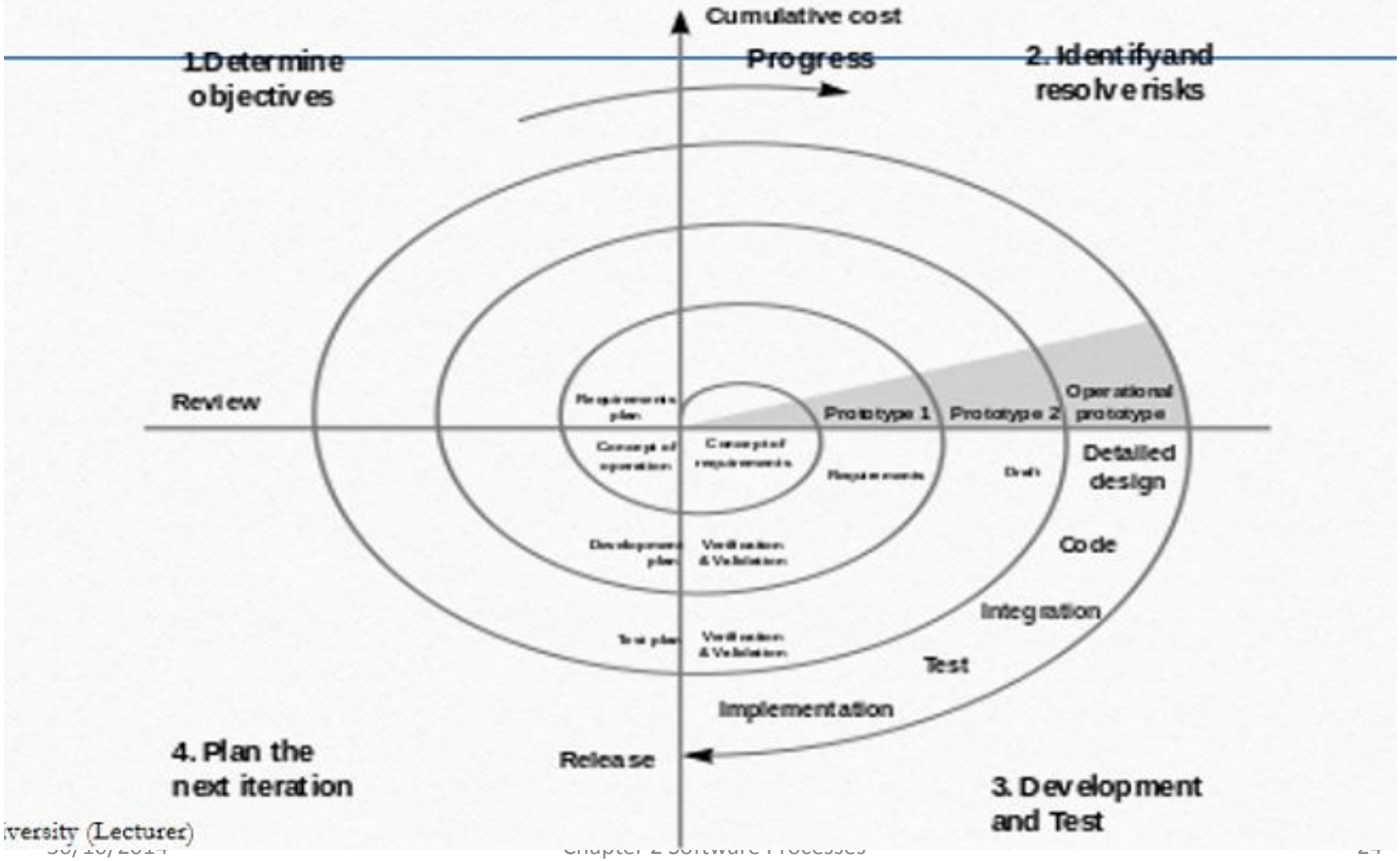
- **Advantages:**
- Requirements are identified in every phase.
- Cost is distributed with less human power & staffing.
- Errors are simultaneously corrected.
- Feedback requirement is clear.
- Testing is easy.
- **Disadvantages:**
- Requires proper planning to distribute the work.
- Total cost required for the development of the product is high.
- Interface model should be well-defined.

The spiral model

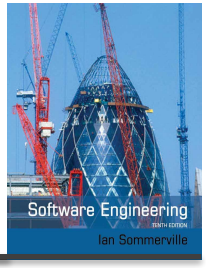


- The **spiral model** is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model.
- It provides the potential for rapid development of increasingly more complete versions of the software.
- The spiral model is a realistic approach to the development of large scale systems and software.
- The spiral development model is a risk-driven process model generator that is used to guide multistakeholder concurrent engineering of software intensive systems

Spiral Model

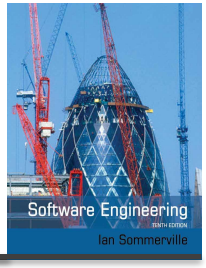


Quadrant–Spiral Model



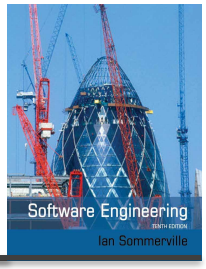
- **First quadrant (Objective Setting)**
- During the first quadrant, it is needed to identify the objectives of the phase.
- Examine the risks associated with these objectives.
- **Second Quadrant (Risk Assessment and Reduction)**
- A detailed analysis is carried out for each identified project risk.
- Steps are taken to reduce the risks.
- For example, if there is a risk that the requirements are inappropriate, a prototype system may be developed

Quadrant–Spiral Model



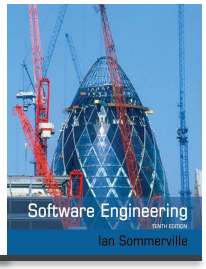
- **Third Quadrant (Development and Validation) •**
- Develop and validate the next level of the product after resolving the identified risks.
- **Fourth Quadrant (Review and Planning)**
- Review the results achieved so far with the customer and plan the next iteration around the spiral.
- Progressively more complete version of the software gets built with each iteration around the spiral.

Circumstances to use spiral model



- The spiral model is called a meta model since it encompasses all other life cycle models. Risk handling is inherently built into this model.
- The spiral model is suitable for development of technically challenging software products that are prone to several kinds of risks.

Advantages and Disadvantages

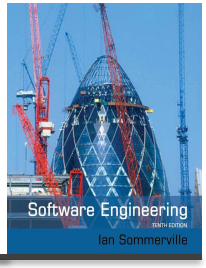


- **Advantage:**
- User will be able to see the project development cycle.
- Risk analysis which resolves higher priority error.
- Project is very much refined.
- Reusability of the software.

Disadvantages:

- It is only suitable for large size project.
- Model is more complex to use.
- Management skill is necessary so as to analyze the risk factor.

The Prototyping Model



- Prototype model focuses on producing software product quickly.
- A prototype paradigm may be the best approach in many situations.
- The developer may be ensured with the efficiency of the algorithm.
- The prototyping model begins with communication.
- In this phase the software engineers and customer must define the overall objective for the software and identify whatever requirements are known, outline the areas where further definition is mandatory.

The Prototyping Model

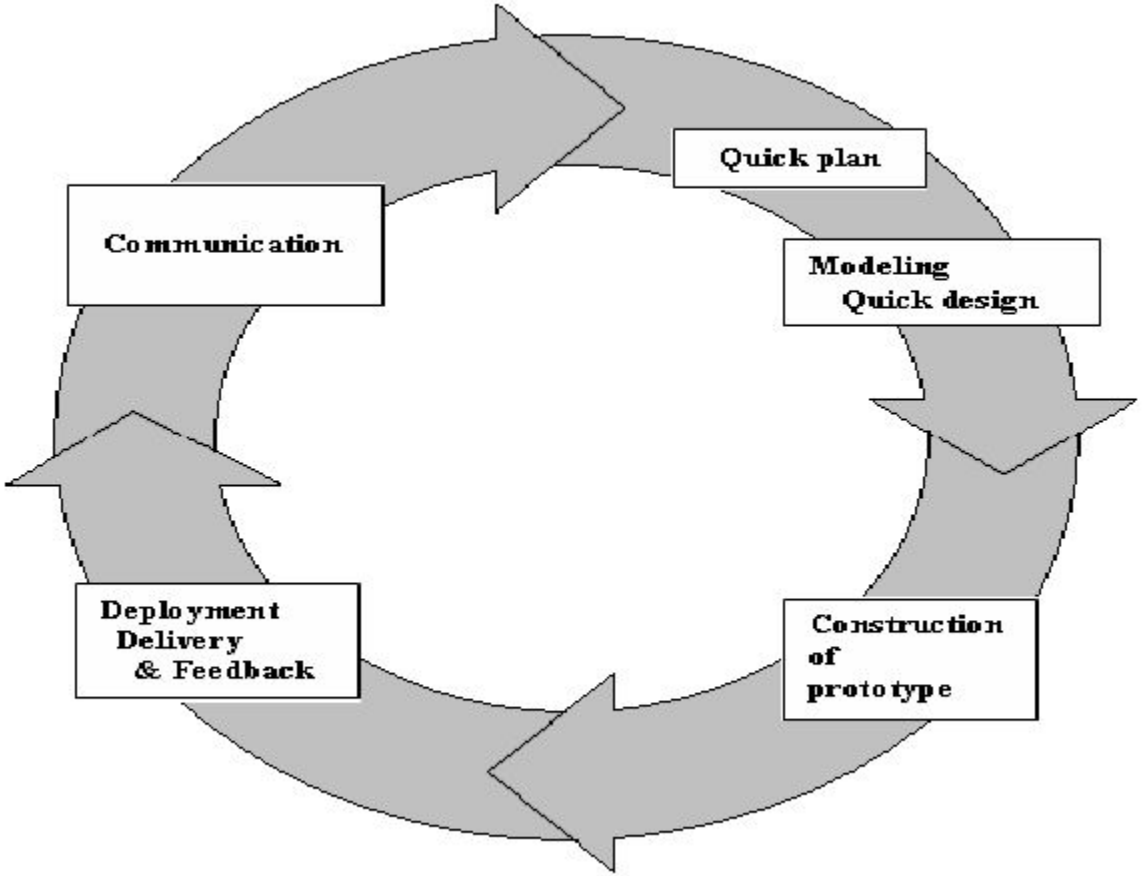
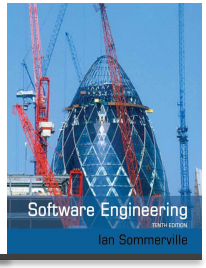


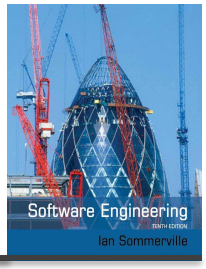
Fig. 1.7. Prototype Model

Problem with the prototyping model



- The customer sees what appears to be a working version of the software product.
- Hence software development management is necessary.
- The developer often makes the implementation quickly which results in the usage of inefficient algorithm.

Advantages and Disadvantages

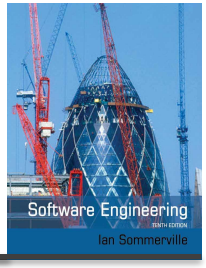


- **Advantage:**
- Iteration process facilitating enhancements.
- Partial product can be viewed by the customer.
- Flexibility of the product.
- Customer satisfaction of the product.

Disadvantage:

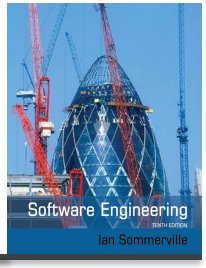
- No optimal solution.
- Time consuming if algorithm used is inefficient.
- Poor documentation resulting with difficulty.

There are four types of model available:



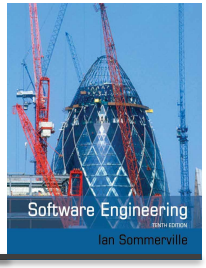
- **A) Rapid Throwaway Prototyping –**
  This technique offers a useful method of exploring ideas and getting customer feedback for each of them. In this method, a developed prototype need not necessarily be a part of the ultimately accepted prototype.
- **B) Evolutionary Prototyping –**
  In this method, the prototype developed initially is incrementally refined on the basis of customer feedback till it finally gets accepted.
- In comparison to Rapid Throwaway Prototyping, it offers a better approach which saves time as well as effort.

There are four types of model available:



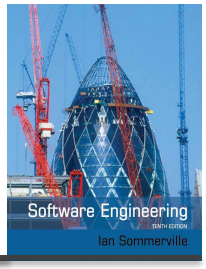
- **C) Incremental Prototyping –** In this type of incremental Prototyping, the final expected product is broken into different small pieces of prototypes and being developed individually.
- In the end, when all individual pieces are properly developed, then the different prototypes are collectively merged into a single final product in their predefined order.
- **D) Extreme Prototyping –** This method is mainly used for web development. It is consists of three sequential independent phases:
- **D.1)** In this phase a basic prototype with all the existing static pages are presented in the HTML format.
- **D.2)** In the 2nd phase, Functional screens are made with a simulate data process using a prototype services layer.
- **D.3)** This is the final step where all the services are implemented and associated with the final prototype.

Software Prototyping - Application



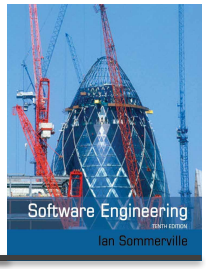
- Software Prototyping is most useful in development of systems having high level of user interactions such as online systems.
- Systems which need users to fill out forms or go through various screens before data is processed can use prototyping very effectively to give the exact look and feel even before the actual software is developed

Evolutionary Models



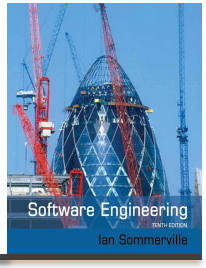
- Iterative + Incremental
- Usually a set of core product or system requirements is well understood, but the details and extension have yet to be defined.
- It is **iterative** that enables you to develop **increasingly** more complete version of the software i.e. evolution of software.
- **Two Types: Prototyping and Spiral**

RAD Model :(Rapid Application Development)



- RAD is an incremental software process models that emphasis a short development cycle
- In RAD model the rapid development is achieved by using a component based construction approach.
- The first phase is communication phase it works to understand the business problem & information characteristics.
- Planning phase is essential because multiple software teams work in parallel on different software function.
-

Drawbacks of RAD:



- For large but scalable projects RAD requires sufficient human resources to create the right number of RAD team.
- If developer and customer are not committed to the rapid-fire activities necessary to complete the software in a much abbreviated time frame, RAD project will fail.
- If a system cannot be properly modularized, building the component necessary for RAD will be problematic.
- If high performance is an issue & performance is to be achieved through tuning the interface to system component, the RAD approach may not work.
- RAD may not be appropriate when technical risks are high. Eg: when a new application makes necessary use of new technology.

RAD

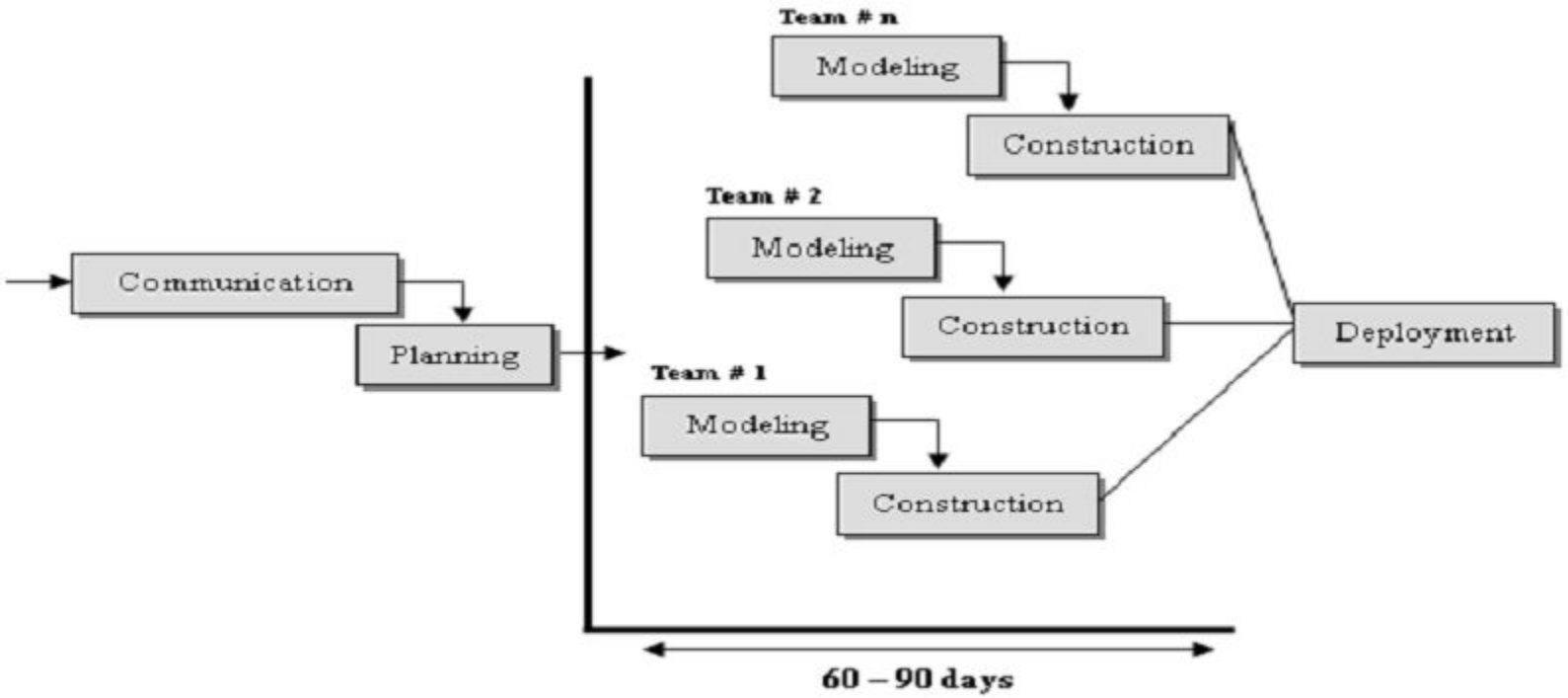
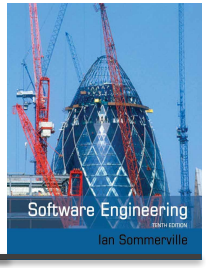
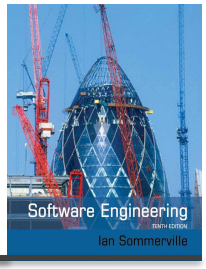


Fig: 1.8 RAD model

Advantages and Disadvantages

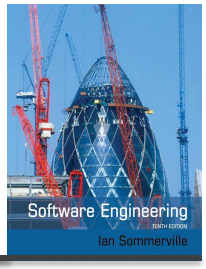


- **Disadvantage:**
- Difficult to divide problem into several unit. □ Used only for very large projects.
  Advantages:
- Reduce errors



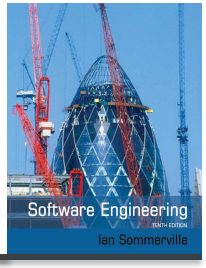
Agile Methodology

Agile Development



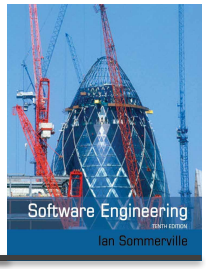
- **Agile development methods** emerged in the late 1990s whose aim was to radically reduce the delivery time for working software systems:
- Program specification, design, and implementation are **interleaved**
- The system is developed as a series of frequent versions or **increments**
- **Stakeholders** involved in version specification and evaluation
- Extensive **tool support** (e.g. automated testing tools) used to support development
- **Minimal documentation** - focus on working code

Agile Manifesto



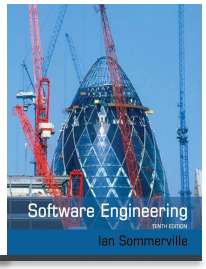
- A team of software developers published the Agile Manifesto in 2001, highlighting the importance of the development team, accommodating changing requirements and customer involvement.
- **Individuals and interactions** over processes and tools.
  - *Mostly focus on team members and their communication not on processes and tools,*
- **Working software** over comprehensive documentation.
  - *Focus on working mechanism of a software not on documentation*
- **Customer collaboration** over contract negotiation.
  - *Focus on customers input more not on contract points*
- **Responding to change** over following a plan.
  - *Implement changes with immediate effect ignore plan.*

The **principles** of agile methods:



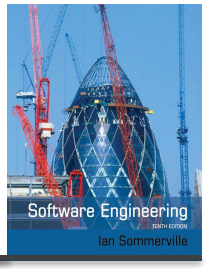
- **Customer involvement**
- Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
- **Incremental delivery**
- The software is developed in increments with the customer specifying the requirements to be included in each increment.
- **People not process**
- The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.

The **principles** of agile methods:



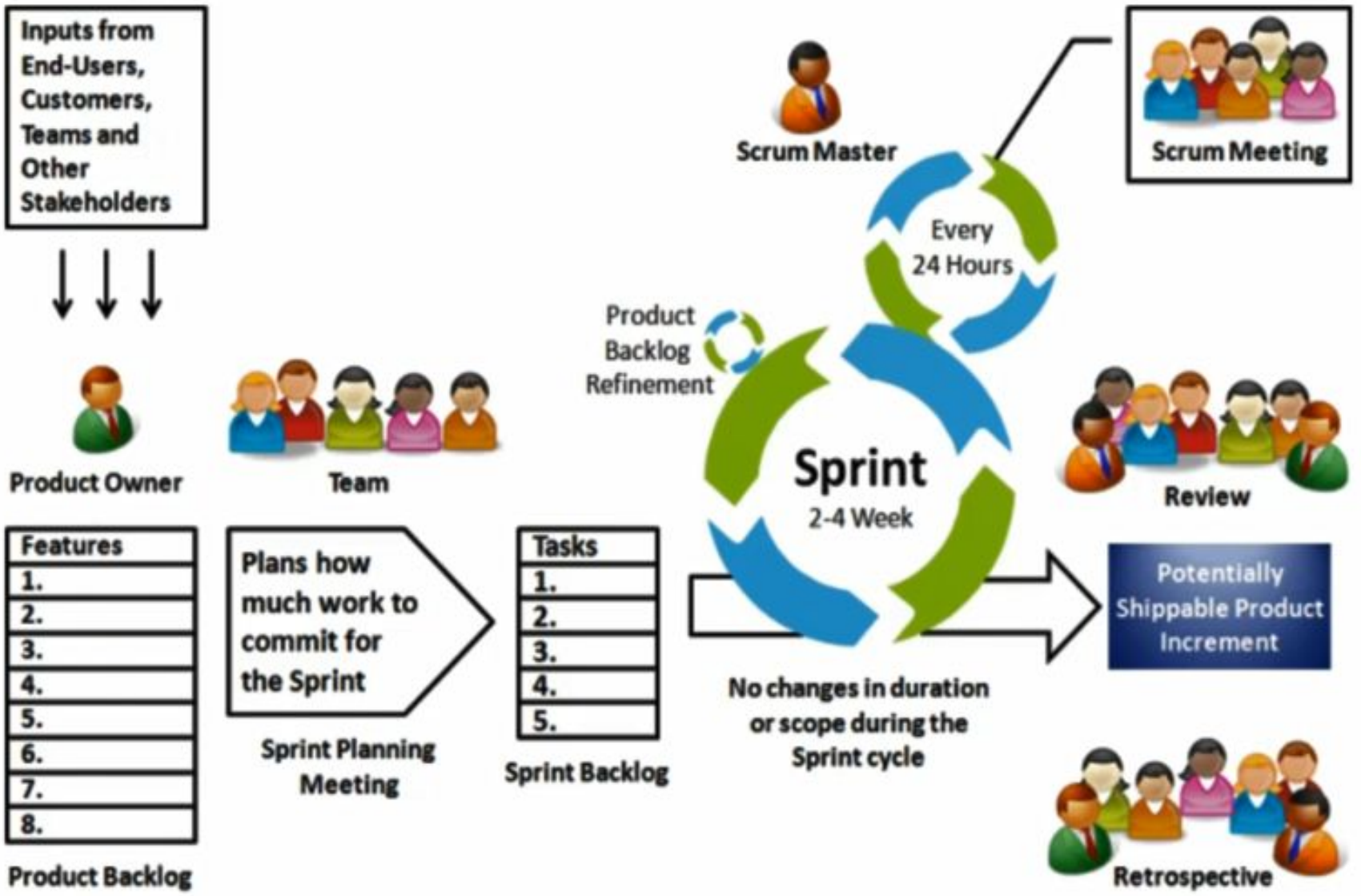
- **Embrace change**
- Expect the system requirements to change and so design the system to accommodate these changes.
- **Maintain simplicity**
- Focus on simplicity in both the software being developed and in the development process.
- Wherever possible, actively work to eliminate complexity from the system.

Scrum Process Model



- The Scrum approach is a general agile method but its focus is on managing iterative development rather than specific agile practices. There are three phases in Scrum:
- The initial phase is an outline planning phase where you establish the general objectives for the project and design the software architecture.
- This is followed by a series of **sprint** cycles, where each cycle develops an increment of the system.
- The project closure phase wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.

SCRUM


Inputs from End-Users, Customers, Teams and Other Stakeholders
Product Owner
Team
Features
1.
2.
3.
4.
5.
6.
7.
8.
Product Backlog
Plans how much work to commit for the Sprint
Sprint Planning Meeting
Tasks
1.
2.
3.
4.
5.
Sprint Backlog
Scrum Master
Scrum Meeting
Every 24 Hours
Product Backlog Refinement
Sprint
2-4 Week
Review
Potentially Shippable Product Increment
No changes in duration or scope during the Sprint cycle
Retrospective

Components of Scrum

- Scrum Roles
- The Process
- Scrum Artifacts

Scrum Master

- Represents management to the project
- Typically filled by a Project Manager or Team Leader
- Responsible for enacting scrum values and practices
- Main job is to remove impediments

The Scrum Team

- Typically 3-9 people
- Cross-functional (QA, Programmers, UI Designers, etc.)
- Members should be full-time
- Team is self-organizing

Product Owner

- Acts like one voice (in any case)
- Knows what needs to be build and in what sequence this should be done
- Typically a product manager

The Process

- Sprint Planning Meeting
- Sprint
- Daily Scrum
- Sprint Review Meeting

Sprint

- A month-long iteration, during which is incremented a product functionality
- NO outside influence can interference with the Scrum team during the Sprint
- Each Sprint begins with the Daily Scrum Meeting

Daily Scrum

- Is a short (15 minutes long) meeting, which is held every day before the Team starts working
- Participants: Scrum Master (which is the chairman), Scrum Team
- "Chickens" and "Pigs"
- Every Team member should answer on 3 questions

Sprint Review Meeting

- Is held at the end of each Sprint
- Business functionality which was created during the Sprint is demonstrated to the Product Owner
- Informal, should not distract Team members of doing their work

Scrum Artifacts

- Product Backlog
- Sprint Backlog
- Burn down Charts

Product Backlog

- Requirements for a system, expressed as a prioritized list of Backlog Items
- Is managed and owned by a Product Owner
- Spreadsheet (typically)
- Usually is created during the Sprint Planning Meeting
- Can be changed and re-prioritized before each PM

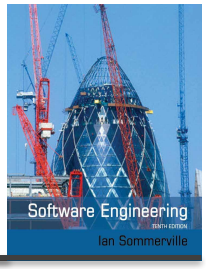
Sprint Backlog

- A subset of Product Backlog Items, which define the work for a Sprint
- Is created ONLY by Team members
- Each Item has it's own status
- Should be updated every day

Sprint Backlog

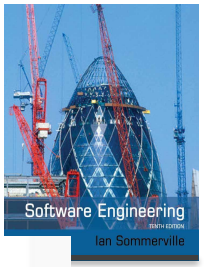
- No more then 300 tasks in the list
- If a task requires more than 16 hours, it should be broken down
- Team can add or subtract items from the list. Product Owner is not allowed to do it

XP(Extreme Programming)



- XP is a lightweight, efficient, low-risk, flexible, predictable, scientific, and fun way to develop a software.
- e**X**treme **P**rogramming (XP) was conceived and developed to address the specific needs of software development by small teams in the face of vague and changing requirements.
- Embrace Change

XP

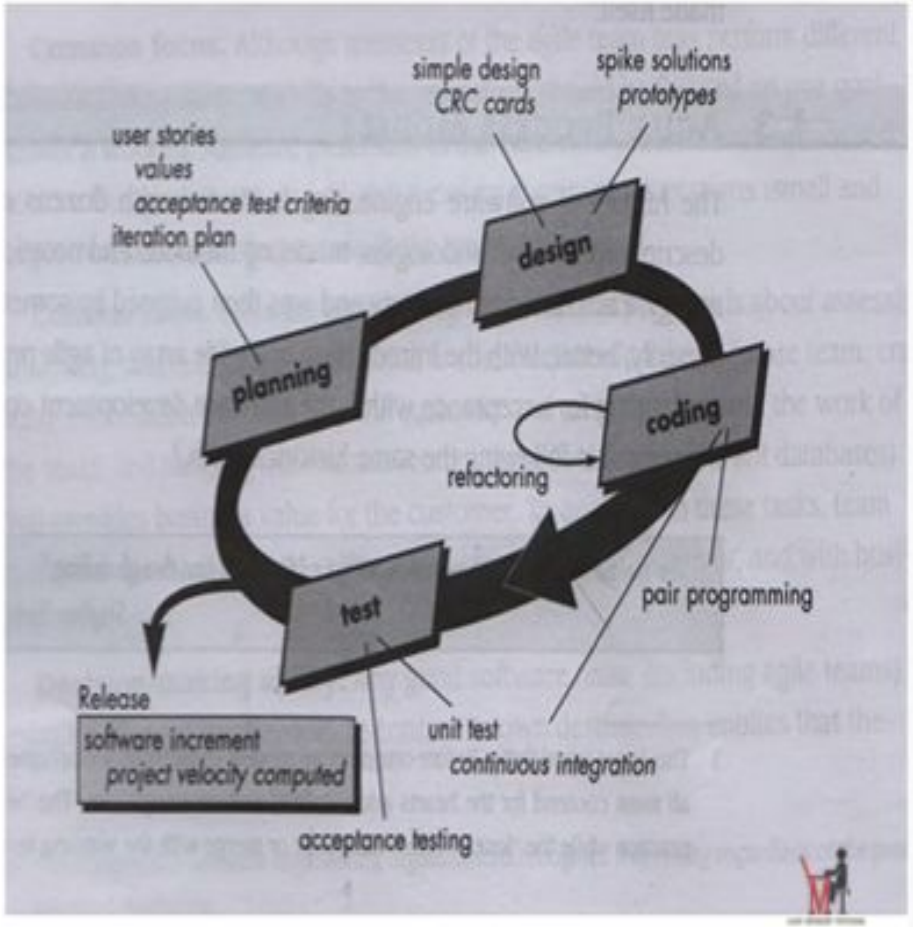


- **EXTREME PROGRAMMING(XP)**

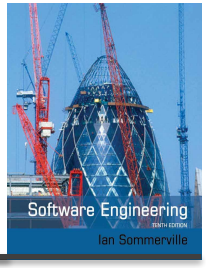
- XP is a lightweight , efficient, low-risk, flexible, predictable, scientific, and fun way to develop a software.
- Small to medium sized teams that work under vague and rapidly changing requirements.
- The five values of XP are communication, simplicity, feedback, courage, and respect
- Follows Object Oriented approach.

Spike solution:A **spike solution** is a very simple program to explore potential **solutions.** Difficult designs should be modeled using prototype.

Refactoring: Improves the internal structure of the code but external behavior not affected.

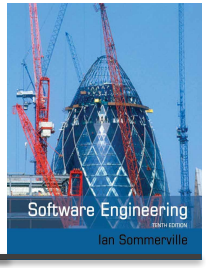


This is how XP supports agile principles:



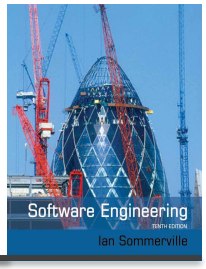
- Incremental development is supported through **small, frequent system releases**.
- Customer involvement means **full-time customer engagement** with the team.
- People not process through **pair programming**, **collective ownership**, and a process that avoids long working hours.
- Change supported through **regular system releases**.
- Maintaining simplicity through **constant refactoring** of code.

Select process model for following



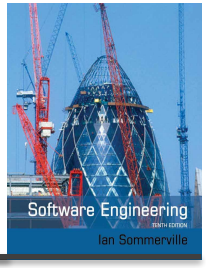
- *A system to control anti-lock braking in a car.*
- *A virtual reality system to support software maintenance.*
- *A university accounting system that replaces an existing system.*
- *An interactive travel planning system that helps users plan journeys with the lowest environmental impact.*

Solutions



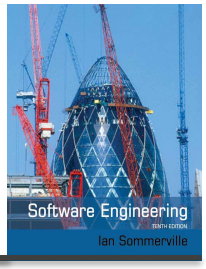
- *a) Anti-lock braking system*
- This is a safety-critical system so requires a lot of up-front analysis before implementation.
- It certainly needs a plan-driven approach to development with the requirements carefully analyzed.
- A waterfall model is therefore the most appropriate approach to use, perhaps with formal transformations between the different development stages.

Solution



- *b) Virtual reality system*
- This is a system where the requirements will change and there will be an extensive user interface components. Incremental development with, perhaps, some UI prototyping is the most appropriate model. An agile process may be used.
- *c) University accounting system*
- This is a system whose requirements are fairly well-known and which will be used in an environment in conjunction with lots of other systems such as a research grant management system.
- Therefore, water fall likely to be appropriate for this.

Solution



- *d) Interactive travel planning system*
- System with a complex user interface but which must be stable and reliable.
- An incremental development approach is the most appropriate as the system requirements will change as real user experience with the system is gained.